

TORCHVISION.TRANSFORMS

Transforms are common image transformations. They can be chained together using `Compose` . Additionally, there is the `torchvision.transforms.functional` module. Functional transforms give fine-grained control over the transformations. This is useful if you have to build a more complex transformation pipeline (e.g. in the case of segmentation tasks).

All transformations accept PIL Image, Tensor Image or batch of Tensor Images as input. Tensor Image is a tensor with `(C, H, W)` shape, where `C` is a number of channels, `H` and `W` are image height and width. Batch of Tensor Images is a tensor of `(B, C, H, W)` shape, where `B` is a number of images in the batch. Deterministic or random transformations applied on the batch of Tensor Images identically transform all the images of the batch.

• WARNING

Since v0.8.0 all random transformations are using torch default random generator to sample random parameters. It is a backward compatibility breaking change and user should set the random state as following:

```
# Previous versions
# import random
# random.seed(12)

# Now
import torch
torch.manual_seed(17)
```

Please, keep in mind that the same seed for torch random generator and Python random generator will not produce the same results.

Scriptable transforms

In order to script the transformations, please use `torch.nn.Sequential` instead of `Compose` .

```
transforms = torch.nn.Sequential(
    transforms.CenterCrop(10),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
)
scripted_transforms = torch.jit.script(transforms)
```

Make sure to use only scriptable transformations, i.e. that work with `torch.Tensor` and does not require *lambda* functions or `PIL.Image` .

For any custom transformations to be used with `torch.jit.script` , they should be derived from `torch.nn.Module` .

Compositions of transforms

CLASS `torchvision.transforms.Compose(transforms)`

[SOURCE]

Composes several transforms together. This transform does not support torchscript. Please, see the note below.

Parameters: `transforms` (list of `Transform` objects) – list of transforms to compose.

Example

```
>>> transforms.Compose([
>>>     transforms.CenterCrop(10),
>>>     transforms.ToTensor(),
>>> ])
```

• NOTE

In order to script the transformations, please use `torch.nn.Sequential` as below.

```
>>> transforms = torch.nn.Sequential(
>>>     transforms.CenterCrop(10),
>>>     transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
>>> )
>>> scripted_transforms = torch.jit.script(transforms)
```

Make sure to use only scriptable transformations, i.e. that work with `torch.Tensor` , does not require *lambda* functions or `PIL.Image` .

Transforms on PIL Image and torch.*Tensor

CLASS

torchvision.transforms.CenterCrop(size)

[SOURCE]

Crops the given image at the center. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions. If image size is smaller than output size along any edge, image is padded with 0 and then center cropped.

Parameters: **size** (sequence or *int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).

forward(img)

[SOURCE]

Parameters: **img** (PIL Image or Tensor) – Image to be cropped.

Returns: Cropped image.

Return type: PIL Image or Tensor

CLASS

torchvision.transforms.ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)

[SOURCE]

Randomly change the brightness, contrast, saturation and hue of an image. If the image is torch Tensor, it is expected to have [..., 3, H, W] shape, where ... means an arbitrary number of leading dimensions. If img is PIL Image, mode “1”, “L”, “I”, “F” and modes with transparency (alpha channel) are not supported.

Parameters:

- brightness** (*float* or tuple of python:float (min, max)) – How much to jitter brightness. brightness_factor is chosen uniformly from [max(0, 1 - brightness), 1 + brightness] or the given [min, max]. Should be non negative numbers.
- contrast** (*float* or tuple of python:float (min, max)) – How much to jitter contrast. contrast_factor is chosen uniformly from [max(0, 1 - contrast), 1 + contrast] or the given [min, max]. Should be non negative numbers.
- saturation** (*float* or tuple of python:float (min, max)) – How much to jitter saturation. saturation_factor is chosen uniformly from [max(0, 1 - saturation), 1 + saturation] or the given [min, max]. Should be non negative numbers.
- hue** (*float* or tuple of python:float (min, max)) – How much to jitter hue. hue_factor is chosen uniformly from [-hue, hue] or the given [min, max]. Should have 0<= hue <= 0.5 or -0.5 <= min <= max <= 0.5.

forward(img)

[SOURCE]

Parameters: **img** (PIL Image or Tensor) – Input image.

Returns: Color jittered image.

Return type: PIL Image or Tensor

STATIC

get_params(brightness: Union[List[float], NoneType], contrast: Union[List[float], NoneType], saturation: Union[List[float], NoneType], hue: Union[List[float], NoneType]) → Tuple[torch.Tensor, Union[float, NoneType], Union[float, NoneType], Union[float, NoneType], Union[float, NoneType]]

[SOURCE]

Get the parameters for the randomized transform to be applied on image.

Parameters:

- brightness** (tuple of python:float (min, max), optional) – The range from which the brightness_factor is chosen uniformly. Pass None to turn off the transformation.
- contrast** (tuple of python:float (min, max), optional) – The range from which the contrast_factor is chosen uniformly. Pass None to turn off the transformation.
- saturation** (tuple of python:float (min, max), optional) – The range from which the saturation_factor is chosen uniformly. Pass None to turn off the transformation.
- hue** (tuple of python:float (min, max), optional) – The range from which the hue_factor is chosen uniformly. Pass None to turn off the transformation.

Returns: The parameters used to apply the randomized transform along with their random order.

Return type: tuple

CLASS

torchvision.transforms.FiveCrop(size)

[SOURCE]

Crop the given image into four corners and the central crop. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

• NOTE

This transform returns a tuple of images and there may be a mismatch in the number of inputs and targets your Dataset returns. See below for an example of how to deal with this.

Parameters: **size** (sequence or *int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop of size (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).

Example

```
>>> transform = Compose([
>>>     FiveCrop(size), # this is a list of PIL Images
>>>     Lambda(lambda crops: torch.stack([ToTensor()(crop) for crop in crops])) # returns a 4D tensor
>>> ])
>>> #In your test loop you can do the following:
>>> input, target = batch # input is a 5d tensor, target is 2d
>>> bs, ncrops, c, h, w = input.size()
>>> result = model(input.view(-1, c, h, w)) # fuse batch size and ncrops
>>> result_avg = result.view(bs, ncrops, -1).mean(1) # avg over crops
```

`forward(img)`

[SOURCE]

Parameters:
Returns:

img (*PIL Image or Tensor*) – Image to be cropped.
tuple of 5 images. Image can be PIL Image or Tensor

CLASS `torchvision.transforms.Grayscale(num_output_channels=1)`

[SOURCE]

Convert image to grayscale. If the image is torch Tensor, it is expected to have [..., 3, H, W] shape, where ... means an arbitrary number of leading dimensions

Parameters:
Returns:

num_output_channels (*int*) – (1 or 3) number of channels desired for output image
Grayscale version of the input.

- If `num_output_channels == 1` : returned image is single channel
- If `num_output_channels == 3` : returned image is 3 channel with `r == g == b`

Return type:

PIL Image

`forward(img)`

[SOURCE]

Parameters:
Returns:
Return type:

img (*PIL Image or Tensor*) – Image to be converted to grayscale.
Grayscaled image.
PIL Image or Tensor

CLASS `torchvision.transforms.Pad(padding, fill=0, padding_mode='constant')`

[SOURCE]

Pad the given image on all sides with the given “pad” value. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means at most 2 leading dimensions for mode reflect and symmetric, at most 3 leading dimensions for mode edge, and an arbitrary number of leading dimensions for mode constant

Parameters:

- **padding** (*int or sequence*) – Padding on each border. If a single int is provided this is used to pad all borders. If sequence of length 2 is provided this is the padding on left/right and top/bottom respectively. If a sequence of length 4 is provided this is the padding for the left, top, right and bottom borders respectively. In torchscript mode padding as single int is not supported, use a sequence of length 1: `[padding, ]`.
- **fill** (*number or str or tuple*) – Pixel fill value for constant fill. Default is 0. If a tuple of length 3, it is used to fill R, G, B channels respectively. This value is only used when the padding_mode is constant. Only number is supported for torch Tensor. Only int or str or tuple value is supported for PIL Image.
- **padding_mode** (*str*) – Type of padding. Should be: constant, edge, reflect or symmetric. Default is constant.
 - constant: pads with a constant value, this value is specified with fill
 - edge: pads with the last value at the edge of the image,

if input a 5D torch Tensor, the last 3 dimensions will be padded instead of the last 2
 - reflect: pads with reflection of image without repeating the last value on the edge

For example, padding [1, 2, 3, 4] with 2 elements on both sides in reflect mode will result in [3, 2, 1, 2, 3, 4, 3, 2]
 - symmetric: pads with reflection of image repeating the last value on the edge

For example, padding [1, 2, 3, 4] with 2 elements on both sides in symmetric mode will result in [2, 1, 1, 2, 3, 4, 4, 3]

`forward(img)`

[SOURCE]

Parameters:
Returns:
Return type:

img (*PIL Image or Tensor*) – Image to be padded.
Padded image.
PIL Image or Tensor

CLASS `torchvision.transforms.RandomAffine(degrees, translate=None, scale=None, shear=None, interpolation=<InterpolationMode.NEAREST: 'nearest'>, fill=0, fillcolor=None, resample=None)`

[SOURCE]

Random affine transformation of the image keeping center invariant. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **degrees** (*sequence or number*) – Range of degrees to select from. If degrees is a number instead of sequence like (min, max), the range of degrees will be (-degrees, +degrees). Set to 0 to deactivate rotations.
 - **translate** (*tuple, optional*) – tuple of maximum absolute fraction for horizontal and vertical translations. For example translate=(a, b), then horizontal shift is randomly sampled in the range -img_width * a < dx < img_width * a and vertical shift is randomly sampled in the range -img_height * b < dy < img_height * b. Will not translate by default.
 - **scale** (*tuple, optional*) – scaling factor interval, e.g (a, b), then scale is randomly sampled from the range a <= scale <= b. Will keep original scale by default.
 - **shear** (*sequence or number, optional*) – Range of degrees to select from. If shear is a number, a shear parallel to the x axis in the range (-shear, +shear) will be applied. Else if shear is a sequence of 2 values a shear parallel to the x axis in the range (shear[0], shear[1]) will be applied. Else if shear is a sequence of 4 values, a x-axis shear in (shear[0], shear[1]) and y-axis shear in (shear[2], shear[3]) will be applied. Will not apply shear by default.
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by `torchvision.transforms.InterpolationMode` . Default is `InterpolationMode.NEAREST` . If input is Tensor, only `InterpolationMode.NEAREST` , `InterpolationMode.BILINEAR` are supported. For backward compatibility integer values (e.g. `PIL.Image.NEAREST`) are still acceptable.
 - **fill** (*sequence or number*) – Pixel fill value for the area outside the transformed image. Default is `0` . If given a number, the value is used for all bands respectively. If input is PIL Image, the options is only available for `Pillow>=5.0.0` .
 - **fillcolor** (*sequence or number, optional*) – deprecated argument and will be removed since v0.10.0. Please use the `fill` parameter instead.
 - **resample** (*int, optional*) – deprecated argument and will be removed since v0.10.0. Please use the `interpolation` parameter instead.

forward(*img*)

[SOURCE]

img (PIL Image or Tensor): Image to be transformed.

Returns:

Affine transformed image.

Return type:

PIL Image or Tensor

STATIC `get_params(degrees: List[float], translate: Union[List[float], NoneType], scale_ranges: Union[List[float], NoneType], shears: Union[List[float], NoneType], img_size: List[int])` → Tuple[float, Tuple[int, int], float, Tuple[float, float]]

[SOURCE]

Get parameters for affine transformation

Returns:

params to be passed to the affine transformation

CLASS `torchvision.transforms.RandomApply(transforms, p=0.5)`

[SOURCE]

Apply randomly a list of transformations with a given probability.

• NOTE

In order to script the transformation, please use `torch.nn.ModuleList` as input instead of list/tuple of transforms as shown below:

>>> transforms = transforms.RandomApply(torch.nn.ModuleList([>>> transforms.ColorJitter(),>>>]), p=0.3)>>> scripted_transforms = torch.jit.script(transforms)

Make sure to use only scriptable transformations, i.e. that work with `torch.Tensor` , does not require *lambda* functions or `PIL.Image` .

- Parameters:
- **transforms** (*sequence or torch.nn.Module*) – list of transformations
 - **p** (*float*) – probability

CLASS `torchvision.transforms.RandomCrop(size, padding=None, pad_if_needed=False, fill=0, padding_mode='constant')`

[SOURCE]

Crop the given image at a random location. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions, but if non-constant padding is used, the input is expected to have at most 2 leading dimensions

- Parameters:
- **size** (*sequence or int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).
 - **padding** (*int or sequence, optional*) – Optional padding on each border of the image. Default is None. If a single int is provided this is used to pad all borders. If sequence of length 2 is provided this is the padding on left/right and top/bottom respectively. If a sequence of length 4 is provided this is the padding for the left, top, right and bottom borders respectively. In torchscript mode padding as single int is not supported, use a sequence of length 1: `[padding, ]`.
 - **pad_if_needed** (*boolean*) – It will pad the image if smaller than the desired size to avoid raising an exception. Since cropping is done after padding, the padding seems to be done at a random offset.
 - **fill** (*number or str or tuple*) – Pixel fill value for constant fill. Default is 0. If a tuple of length 3, it is used to fill R, G, B channels respectively. This value is only used when the padding_mode is constant. Only number is supported for torch Tensor. Only int or str or tuple value is supported for PIL Image.
 - **padding_mode** (*str*) –
Type of padding. Should be: constant, edge, reflect or symmetric. Default is constant.
 - constant: pads with a constant value, this value is specified with fill
 - edge: pads with the last value on the edge of the image
 - reflect: pads with reflection of image (without repeating the last value on the edge)

padding [1, 2, 3, 4] with 2 elements on both sides in reflect mode will result in [3, 2, 1, 2, 3, 4, 3, 2]
 - symmetric: pads with reflection of image (repeating the last value on the edge)

padding [1, 2, 3, 4] with 2 elements on both sides in symmetric mode will result in [2, 1, 1, 2, 3, 4, 4, 3]

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be cropped.

Returns:

Cropped image.

Return type:

PIL Image or Tensor

STATIC get_params(*img: torch.Tensor, output_size: Tuple[int, int]*) → Tuple[int, int, int, int]

[SOURCE]

Get parameters for `crop` for a random crop.

Parameters:

- **img** (*PIL Image or Tensor*) – Image to be cropped.
- **output_size** (*tuple*) – Expected output size of the crop.

Returns:

params (i, j, h, w) to be passed to `crop` for random crop.

Return type:

tuple

CLASS torchvision.transforms.RandomGrayscale(*p=0.1*)

[SOURCE]

Randomly convert image to grayscale with a probability of p (default 0.1). If the image is torch Tensor, it is expected to have [..., 3, H, W] shape, where ... means an arbitrary number of leading dimensions

Parameters:

p (*float*) – probability that image should be converted to grayscale.

Returns:

Grayscale version of the input image with probability p and unchanged with probability (1-p). - If input image is 1 channel: grayscale version is 1 channel - If input image is 3 channel: grayscale version is 3 channel with r == g == b

Return type:

PIL Image or Tensor

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be converted to grayscale.

Returns:

Randomly grayscaled image.

Return type:

PIL Image or Tensor

CLASS torchvision.transforms.RandomHorizontalFlip(*p=0.5*)

[SOURCE]

Horizontally flip the given image randomly with a given probability. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

Parameters:

p (*float*) – probability of the image being flipped. Default value is 0.5

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be flipped.

Returns:

Randomly flipped image.

Return type:

PIL Image or Tensor

CLASS torchvision.transforms.RandomPerspective(*distortion_scale=0.5, p=0.5, interpolation=<InterpolationMode.BILINEAR: 'bilinear'>, fill=0*)

[SOURCE]

Performs a random perspective transformation of the given image with a given probability. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **distortion_scale** (*float*) – argument to control the degree of distortion and ranges from 0 to 1. Default is 0.5.
 - **p** (*float*) – probability of the image being transformed. Default is 0.5.
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by `torchvision.transforms.InterpolationMode` . Default is `InterpolationMode.BILINEAR` . If input is Tensor, only `InterpolationMode.NEAREST` , `InterpolationMode.BILINEAR` are supported. For backward compatibility integer values (e.g. `PIL.Image.NEAREST`) are still acceptable.
 - **fill** (*sequence or number*) – Pixel fill value for the area outside the transformed image. Default is `0` . If given a number, the value is used for all bands respectively. If input is PIL Image, the options is only available for `Pillow>=5.0.0` .

`forward(img)`

[SOURCE]

Parameters: **img** (*PIL Image or Tensor*) – Image to be Perspectively transformed.

Returns: Randomly transformed image.

Return type: PIL Image or Tensor

STATIC `get_params(width: int, height: int, distortion_scale: float) → Tuple[List[List[int]], List[List[int]]]`

[SOURCE]

Get parameters for `perspective` for a random perspective transform.

Parameters:

- **width** (*int*) – width of the image.
- **height** (*int*) – height of the image.
- **distortion_scale** (*float*) – argument to control the degree of distortion and ranges from 0 to 1.

Returns: List containing [top-left, top-right, bottom-right, bottom-left] of the original image, List containing [top-left, top-right, bottom-right, bottom-left] of the transformed image.

CLASS `torchvision.transforms.RandomResizedCrop(size, scale=(0.08, 1.0), ratio=(0.75, 1.3333333333333333), interpolation=<InterpolationMode.BILINEAR: 'bilinear'>)`

[SOURCE]

Crop the given image to random size and aspect ratio. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

A crop of random size (default: of 0.08 to 1.0) of the original size and a random aspect ratio (default: of 3/4 to 4/3) of the original aspect ratio is made. This crop is finally resized to given size. This is popularly used to train the Inception networks.

- Parameters:
- **size** (*int or sequence*) – expected output size of each edge. If size is an int instead of sequence like (h, w), a square output size (**size**, **size**) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]). In torchscript mode size as single int is not supported, use a sequence of length 1: [**size**,] .
 - **scale** (*tuple of python:float*) – scale range of the cropped image before resizing, relatively to the origin image.
 - **ratio** (*tuple of python:float*) – aspect ratio range of the cropped image before resizing.
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by `torchvision.transforms.InterpolationMode` . Default is `InterpolationMode.BILINEAR` . If input is Tensor, only `InterpolationMode.NEAREST` , `InterpolationMode.BILINEAR` and `InterpolationMode.BICUBIC` are supported. For backward compatibility integer values (e.g. `PIL.Image.NEAREST`) are still acceptable.

`forward(img)`

[SOURCE]

Parameters: **img** (*PIL Image or Tensor*) – Image to be cropped and resized.

Returns: Randomly cropped and resized image.

Return type: PIL Image or Tensor

STATIC `get_params(img: torch.Tensor, scale: List[float], ratio: List[float]) → Tuple[int, int, int, int]`

[SOURCE]

Get parameters for `crop` for a random sized crop.

- Parameters:**
- **img** (*PIL Image or Tensor*) – Input image.
 - **scale** (*list*) – range of scale of the origin size cropped
 - **ratio** (*list*) – range of aspect ratio of the origin aspect ratio cropped

Returns:

`params` (i, j, h, w) to be passed to `crop` for a random sized crop.

Return type: *tuple*

CLASS `torchvision.transforms.RandomRotation(degrees, interpolation=<InterpolationMode.NEAREST: 'nearest'>, expand=False, center=None, fill=0, resample=None)`

[SOURCE]

Rotate the image by angle. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **degrees** (*sequence or number*) – Range of degrees to select from. If degrees is a number instead of sequence like (min, max), the range of degrees will be (-degrees, +degrees).
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by `torchvision.transforms.InterpolationMode`. Default is `InterpolationMode.NEAREST`. If input is Tensor, only `InterpolationMode.NEAREST`, `InterpolationMode.BILINEAR` are supported. For backward compatibility integer values (e.g. `PIL.Image.NEAREST`) are still acceptable.
 - **expand** (*bool, optional*) – Optional expansion flag. If true, expands the output to make it large enough to hold the entire rotated image. If false or omitted, make the output image the same size as the input image. Note that the expand flag assumes rotation around the center and no translation.
 - **center** (*sequence, optional*) – Optional center of rotation, (x, y). Origin is the upper left corner. Default is the center of the image.
 - **fill** (*sequence or number*) – Pixel fill value for the area outside the rotated image. Default is `0`. If given a number, the value is used for all bands respectively. If input is PIL Image, the options is only available for `Pillow>=5.2.0`.
 - **resample** (*int, optional*) – deprecated argument and will be removed since v0.10.0. Please use the `interpolation` parameter instead.

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be rotated.

Returns:

Rotated image.

Return type:

PIL Image or Tensor

STATIC get_params(*degrees: List[float]*) → float

[SOURCE]

Get parameters for `rotate` for a random rotation.

Returns:

angle parameter to be passed to `rotate` for random rotation.

Return type:

float

CLASS torchvision.transforms.RandomSizedCrop(*args, **kwargs)

[SOURCE]

Note: This transform is deprecated in favor of RandomResizedCrop.

CLASS torchvision.transforms.RandomVerticalFlip(p=0.5)

[SOURCE]

Vertically flip the given image randomly with a given probability. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

Parameters:

p (*float*) – probability of the image being flipped. Default value is 0.5

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be flipped.

Returns:

Randomly flipped image.

Return type:

PIL Image or Tensor

CLASS torchvision.transforms.Resize(size, interpolation=<InterpolationMode.BILINEAR: 'bilinear'>)

[SOURCE]

Resize the input image to the given size. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

- Parameters:
- **size** (*sequence or int*) – Desired output size. If size is a sequence like (h, w), output size will be matched to this. If size is an int, smaller edge of the image will be matched to this number. i.e, if height > width, then image will be rescaled to (size * height / width, size). In torchscript mode size as single int is not supported, use a sequence of length 1: `[size, ]`.
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by `torchvision.transforms.InterpolationMode`. Default is `InterpolationMode.BILINEAR`. If input is Tensor, only `InterpolationMode.NEAREST`, `InterpolationMode.BILINEAR` and `InterpolationMode.BICUBIC` are supported. For backward compatibility integer values (e.g. `PIL.Image.NEAREST`) are still acceptable.

forward(*img*)

[SOURCE]

Parameters:

img (*PIL Image or Tensor*) – Image to be scaled.

Returns:

Rescaled image.

Return type:

PIL Image or Tensor

CLASS torchvision.transforms.Scale(*args, **kwargs)

[SOURCE]

Note: This transform is deprecated in favor of Resize.

CLASS torchvision.transforms.TenCrop(size, vertical_flip=False)

[SOURCE]

Crop the given image into four corners and the central crop plus the flipped version of these (horizontal flipping is used by default). If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

• NOTE

This transform returns a tuple of images and there may be a mismatch in the number of inputs and targets your Dataset returns. See below for an example of how to deal with this.

- Parameters:
- **size** (*sequence or int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).
 - **vertical_flip** (*bool*) – Use vertical flipping instead of horizontal

Example

```
>>> transform = Compose([
>>>     TenCrop(size), # this is a list of PIL Images
>>>     Lambda(lambda crops: torch.stack([ToTensor()(crop) for crop in crops])) # returns a 4D tensor
>>> ])
>>> #In your test loop you can do the following:
>>> input, target = batch # input is a 5d tensor, target is 2d
>>> bs, ncrops, c, h, w = input.size()
>>> result = model(input.view(-1, c, h, w)) # fuse batch size and ncrops
>>> result_avg = result.view(bs, ncrops, -1).mean(1) # avg over crops
```

`forward(img)`

[SOURCE]

- Parameters:
- img** (*PIL Image or Tensor*) – Image to be cropped.
- Returns:
- tuple of 10 images. Image can be PIL Image or Tensor

CLASS `torchvision.transforms.GaussianBlur(kernel_size, sigma=(0.1, 2.0))`

[SOURCE]

Blurs image with randomly chosen Gaussian blur. If the image is torch Tensor, it is expected to have [..., C, H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **kernel_size** (*int or sequence*) – Size of the Gaussian kernel.
 - **sigma** (*float or tuple of python:float (min, max)*) – Standard deviation to be used for creating kernel to perform blurring. If float, sigma is fixed. If it is tuple of float (min, max), sigma is chosen uniformly at random to lie in the given range.
- Returns:
- Gaussian blurred version of the input image.
- Return type:
- PIL Image or Tensor

`forward(img: torch.Tensor)` → torch.Tensor

[SOURCE]

- Parameters:
- img** (*PIL Image or Tensor*) – image to be blurred.
- Returns:
- Gaussian blurred image
- Return type:
- PIL Image or Tensor

STATIC `get_params(sigma_min: float, sigma_max: float)` → float

[SOURCE]

Choose sigma for random gaussian blurring.

- Parameters:
- **sigma_min** (*float*) – Minimum standard deviation that can be chosen for blurring kernel.
 - **sigma_max** (*float*) – Maximum standard deviation that can be chosen for blurring kernel.
- Returns:
- Standard deviation to be passed to calculate kernel for gaussian blurring.
- Return type:
- float

Transforms on PIL Image only

CLASS `torchvision.transforms.RandomChoice(transforms)`

[SOURCE]

Apply single transformation randomly picked from a list. This transform does not support torchscript.

CLASS `torchvision.transforms.RandomOrder(transforms)`

[SOURCE]

Apply a list of transformations in a random order. This transform does not support torchscript.

Transforms on torch.*Tensor only

CLASS `torchvision.transforms.LinearTransformation(transformation_matrix, mean_vector)`

[SOURCE]

Transform a tensor image with a square transformation matrix and a mean_vector computed offline. This transform does not support PIL Image. Given transformation_matrix and mean_vector, will flatten the torch.*Tensor and subtract mean_vector from it which is then followed by computing the dot product with the transformation matrix and then reshaping the tensor to its original shape.

Applications:

whitening transformation: Suppose X is a column vector zero-centered data. Then compute the data covariance matrix [D x D] with torch.mm(X.t(), X), perform SVD on this matrix and pass it as transformation_matrix.

- Parameters:
- **transformation_matrix** (*Tensor*) – tensor [D x D], D = C x H x W
 - **mean_vector** (*Tensor*) – tensor [D], D = C x H x W

`forward(tensor: torch.Tensor)` → torch.Tensor

[SOURCE]

- Parameters:
- tensor** (*Tensor*) – Tensor image to be whitened.
- Returns:
- Transformed image.

Return type: Tensor

CLASS torchvision.transforms.Normalize(mean, std, inplace=False) [SOURCE]

Normalize a tensor image with mean and standard deviation. This transform does not support PIL Image. Given mean: (mean[1],...,mean[n]) and std: (std[1],...,std[n]) for n channels, this transform will normalize each channel of the input torch.*Tensor i.e., $\text{output[channel]} = (\text{input[channel]} - \text{mean[channel]}) / \text{std[channel]}$

• NOTE

This transform acts out of place, i.e., it does not mutate the input tensor.

- Parameters:
- **mean** (sequence) – Sequence of means for each channel.
 - **std** (sequence) – Sequence of standard deviations for each channel.
 - **inplace** (bool, optional) – Bool to make this operation in-place.

forward(tensor: torch.Tensor) → torch.Tensor [SOURCE]

Parameters: tensor (Tensor) – Tensor image to be normalized.

Returns: Normalized Tensor image.

Return type: Tensor

CLASS torchvision.transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 3.3), value=0, inplace=False) [SOURCE]

Randomly selects a rectangle region in an torch Tensor image and erases its pixels. This transform does not support PIL Image. ‘Random Erasing Data Augmentation’ by Zhong et al. See <https://arxiv.org/abs/1708.04896>

- Parameters:
- **p** – probability that the random erasing operation will be performed.
 - **scale** – range of proportion of erased area against input image.
 - **ratio** – range of aspect ratio of erased area.
 - **value** – erasing value. Default is 0. If a single int, it is used to erase all pixels. If a tuple of length 3, it is used to erase R, G, B channels respectively. If a str of ‘random’, erasing each pixel with random values.
 - **inplace** – boolean to make this transform inplace. Default set to False.

Returns: Erased Image.

Example

```
>>> transform = transforms.Compose([
>>>     transforms.RandomHorizontalFlip(),
>>>     transforms.ToTensor(),
>>>     transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
>>>     transforms.RandomErasing(),
>>> ])
```

forward(img) [SOURCE]

Parameters: img (Tensor) – Tensor image to be erased.

Returns: Erased Tensor image.

Return type: img (Tensor)

STATIC get_params(img: torch.Tensor, scale: Tuple[float, float], ratio: Tuple[float, float], value: Union[List[float], NoneType] = None) → Tuple[int, int, int, int, torch.Tensor] [SOURCE]

Get parameters for `erase` for a random erasing.

- Parameters:
- **img** (Tensor) – Tensor image to be erased.
 - **scale** (sequence) – range of proportion of erased area against input image.
 - **ratio** (sequence) – range of aspect ratio of erased area.
 - **value** (list, optional) – erasing value. If None, it is interpreted as “random” (erasing each pixel with random values). If `len(value)` is 1, it is interpreted as a number, i.e. `value[0]` .
- Returns: params (i, j, h, w, v) to be passed to `erase` for random erasing.
- Return type: tuple

CLASS torchvision.transforms.ConvertImageDtype(dtype: torch.dtype) → None [SOURCE]

Convert a tensor image to the given dtype and scale the values accordingly This function does not support PIL Image.

Parameters: dtype (torch.dpython:type) – Desired data type of the output

• NOTE

When converting from a smaller to a larger integer dtype the maximum values are **not** mapped exactly. If converted back and forth, this mismatch has no effect.

Raises: `RuntimeError` – When trying to cast `torch.float32` to `torch.int32` or `torch.int64` as well as for trying to cast `torch.float64` to `torch.int64` . These conversions might lead to overflow errors since the floating point `dtype` cannot store consecutive integers over the whole range of the integer `dtype` .

Conversion Transforms

CLASS `torchvision.transforms.ToPILImage(mode=None)` [SOURCE]

Convert a tensor or an ndarray to PIL Image. This transform does not support torchscript.

Converts a torch.*Tensor of shape C x H x W or a numpy ndarray of shape H x W x C to a PIL Image while preserving the value range.

Parameters: **mode** (`PIL.Image mode`) – color space and pixel depth of input data (optional). If `mode` is `None` (default) there are some assumptions made about the input data: - If the input has 4 channels, the `mode` is assumed to be `RGBA` . - If the input has 3 channels, the `mode` is assumed to be `RGB` . - If the input has 2 channels, the `mode` is assumed to be `LA` . - If the input has 1 channel, the `mode` is determined by the data type (i.e `int` , `float` , `short`).

CLASS `torchvision.transforms.ToTensor` [SOURCE]

Convert a `PIL Image` or `numpy.ndarray` to tensor. This transform does not support torchscript.

Converts a `PIL Image` or `numpy.ndarray` (H x W x C) in the range [0, 255] to a `torch.FloatTensor` of shape (C x H x W) in the range [0.0, 1.0] if the `PIL Image` belongs to one of the modes (L, LA, P, I, F, RGB, YCbCr, RGBA, CMYK, 1) or if the `numpy.ndarray` has `dtype = np.uint8`

In the other cases, tensors are returned without scaling.

• NOTE

Because the input image is scaled to [0.0, 1.0], this transformation should not be used when transforming target image masks. See the [references](#) for implementing the transforms for image masks.

Generic Transforms

CLASS `torchvision.transforms.Lambda(lambd)` [SOURCE]

Apply a user-defined lambda as a transform. This transform does not support torchscript.

Parameters: **lambd** (*function*) – Lambda/function to be used for transform.

Functional Transforms

Functional transforms give you fine-grained control of the transformation pipeline. As opposed to the transformations above, functional transforms don’t contain a random number generator for their parameters. That means you have to specify/generate all parameters, but you can reuse the functional transform.

Example: you can apply a functional transform with the same parameters to multiple images like this:

```
import torchvision.transforms.functional as TF
import random

def my_segmentation_transforms(image, segmentation):
    if random.random() > 0.5:
        angle = random.randint(-30, 30)
        image = TF.rotate(image, angle)
        segmentation = TF.rotate(segmentation, angle)
    # more transforms ...
    return image, segmentation
```

Example: you can use a functional transform to build transform classes with custom behavior:

```
import torchvision.transforms.functional as TF
import random

class MyRotationTransform:
    """Rotate by one of the given angles."""

    def __init__(self, angles):
        self.angles = angles

    def __call__(self, x):
        angle = random.choice(self.angles)
        return TF.rotate(x, angle)

rotation_transform = MyRotationTransform(angles=[-30, -15, 0, 15, 30])
```

CLASS `torchvision.transforms.functional.InterpolationMode` [SOURCE]

Interpolation modes

`torchvision.transforms.functional.adjust_brightness(img: torch.Tensor, brightness_factor: float) → torch.Tensor` [SOURCE]

Adjust brightness of an image.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • brightness_factor (<i>float</i>) – How much to adjust the brightness. Can be any non negative number. 0 gives a black image, 1 gives the original image while 2 increases the brightness by a factor of 2.
Returns:	Brightness adjusted image.
Return type:	PIL Image or Tensor

`torchvision.transforms.functional.adjust_contrast(img: torch.Tensor, contrast_factor: float) → torch.Tensor`

[SOURCE]

Adjust contrast of an image.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • contrast_factor (<i>float</i>) – How much to adjust the contrast. Can be any non negative number. 0 gives a solid gray image, 1 gives the original image while 2 increases the contrast by a factor of 2.
Returns:	Contrast adjusted image.
Return type:	PIL Image or Tensor

`torchvision.transforms.functional.adjust_gamma(img: torch.Tensor, gamma: float, gain: float = 1) → torch.Tensor`

[SOURCE]

Perform gamma correction on an image.

Also known as Power Law Transform. Intensities in RGB mode are adjusted based on the following equation:

$$I_{\text{out}} = 255 \times \text{gain} \times \left(\frac{I_{\text{in}}}{255} \right)^\gamma$$

See [Gamma Correction](#) for more details.

Parameters:	• img (<i>PIL Image or Tensor</i>) – PIL Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • img is PIL Image, modes with transparency (If) – • gamma (<i>float</i>) – Non negative real number, same as γ in the equation. gamma larger than 1 make the shadows darker, while gamma smaller than 1 make dark regions lighter. • gain (<i>float</i>) – The constant multiplier.
Returns:	Gamma correction adjusted image.
Return type:	PIL Image or Tensor

`torchvision.transforms.functional.adjust_hue(img: torch.Tensor, hue_factor: float) → torch.Tensor`

[SOURCE]

Adjust hue of an image.

The image hue is adjusted by converting the image to HSV and cyclically shifting the intensities in the hue channel (H). The image is then converted back to original image mode.

hue_factor is the amount of shift in H channel and must be in the interval [-0.5, 0.5].

See [Hue](#) for more details.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • img is PIL Image mode "1", "L", "I", "F" and modes with transparency (If) – • hue_factor (<i>float</i>) – How much to shift the hue channel. Should be in [-0.5, 0.5]. 0.5 and -0.5 give complete reversal of hue channel in HSV space in positive and negative direction respectively. 0 means no shift. Therefore, both -0.5 and 0.5 will give an image with complementary colors while 0 gives the original image.
Returns:	Hue adjusted image.
Return type:	PIL Image or Tensor

`torchvision.transforms.functional.adjust_saturation(img: torch.Tensor, saturation_factor: float) → torch.Tensor`

[SOURCE]

Adjust color saturation of an image.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • saturation_factor (<i>float</i>) – How much to adjust the saturation. 0 will give a black and white image, 1 will give the original image while 2 will enhance the saturation by a factor of 2.
Returns:	Saturation adjusted image.
Return type:	PIL Image or Tensor

`torchvision.transforms.functional.adjust_sharpness(img: torch.Tensor, sharpness_factor: float) → torch.Tensor`

[SOURCE]

Adjust the sharpness of an image.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be adjusted. • img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, (If) – • ... means it can have an arbitrary number of leading dimensions. (where) – • sharpness_factor (<i>float</i>) – How much to adjust the sharpness. Can be any non negative number. 0 gives a blurred image, 1 gives the original image while 2 increases the sharpness by a factor of 2.
Returns:	Sharpness adjusted image.
Return type:	PIL Image or Tensor

```
torchvision.transforms.functional.affine(img: torch.Tensor, angle: float, translate: List[int], scale: float, shear: List[float], interpolation: torchvision.transforms.functional.InterpolationMode = <InterpolationMode.NEAREST: 'nearest'>, fill: Union[List[float], NoneType] = None, resample: Union[int, NoneType] = None, fillcolor: Union[List[float], NoneType] = None) → torch.Tensor
```

[SOURCE]

Apply affine transformation on the image keeping image center invariant. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

Parameters:	• img (<i>PIL Image or Tensor</i>) – image to transform. • angle (<i>number</i>) – rotation angle in degrees between -180 and 180, clockwise direction. • translate (<i>sequence of python:integers</i>) – horizontal and vertical translations (post-rotation translation) • scale (<i>float</i>) – overall scale • shear (<i>float or sequence</i>) – shear angle value in degrees between -180 to 180, clockwise direction. If a sequence is specified, the first value corresponds to a shear parallel to the x axis, while the second value corresponds to a shear parallel to the y axis. • interpolation (<i>InterpolationMode</i>) – Desired interpolation enum defined by <code>torchvision.transforms.InterpolationMode</code> . Default is <code>InterpolationMode.NEAREST</code> . If input is Tensor, only <code>InterpolationMode.NEAREST</code> , <code>InterpolationMode.BILINEAR</code> are supported. For backward compatibility integer values (e.g. <code>PIL.Image.NEAREST</code>) are still acceptable. • fill (<i>sequence or number, optional</i>) – Pixel fill value for the area outside the transformed image. If given a number, the value is used for all bands respectively. In torchscript mode single int/float value is not supported, please use a sequence of length 1: <code>[value,]</code> . If input is PIL Image, the options is only available for <code>Pillow>=5.0.0</code> . • fillcolor (<i>sequence, int, float</i>) – deprecated argument and will be removed since v0.10.0. Please use the <code>fill</code> parameter instead. • resample (<i>int, optional</i>) – deprecated argument and will be removed since v0.10.0. Please use the <code>interpolation</code> parameter instead.
Returns:	Transformed image.
Return type:	PIL Image or Tensor

```
torchvision.transforms.functional.autocontrast(img: torch.Tensor) → torch.Tensor
```

[SOURCE]

Maximize contrast of an image by remapping its pixels per channel so that the lowest becomes black and the lightest becomes white.

Parameters:	img (<i>PIL Image or Tensor</i>) – Image on which autocontrast is applied. If img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, where ... means it can have an arbitrary number of leading dimensions. If img is PIL Image, it is expected to be in mode “L” or “RGB”.
Returns:	An image that was autocontrasted.
Return type:	PIL Image or Tensor

```
torchvision.transforms.functional.center_crop(img: torch.Tensor, output_size: List[int]) → torch.Tensor
```

[SOURCE]

Crops the given image at the center. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions. If image size is smaller than output size along any edge, image is padded with 0 and then center cropped.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be cropped. • output_size (<i>sequence or int</i>) – (height, width) of the crop box. If int or sequence with single int, it is used for both directions.
Returns:	Cropped image.
Return type:	PIL Image or Tensor

```
torchvision.transforms.functional.convert_image_dtype(image: torch.Tensor, dtype: torch.dtype = torch.float32) → torch.Tensor
```

[SOURCE]

Convert a tensor image to the given dtype and scale the values accordingly This function does not support PIL Image.

Parameters:	• image (<i>torch.Tensor</i>) – Image to be converted • dtype (<i>torch.dpython:type</i>) – Desired data type of the output
Returns:	Converted image
Return type:	Tensor

• NOTE

When converting from a smaller to a larger integer dtype the maximum values are **not** mapped exactly. If converted back and forth, this mismatch has no effect.

Raises:	<code>RuntimeError</code> – When trying to cast <code>torch.float32</code> to <code>torch.int32</code> or <code>torch.int64</code> as well as for trying to cast <code>torch.float64</code> to <code>torch.int64</code> . These conversions might lead to overflow errors since the floating point dtype cannot store consecutive integers over the whole range of the integer dtype .
----------------	--

```
torchvision.transforms.functional.crop(img: torch.Tensor, top: int, left: int, height: int, width: int) → torch.Tensor
```

[SOURCE]

Crop the given image at specified location and output size. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

- Parameters:
- **img** (*PIL Image or Tensor*) – Image to be cropped. (0,0) denotes the top left corner of the image.
 - **top** (*int*) – Vertical component of the top left corner of the crop box.
 - **left** (*int*) – Horizontal component of the top left corner of the crop box.
 - **height** (*int*) – Height of the crop box.
 - **width** (*int*) – Width of the crop box.

Returns: Cropped image.

Return type: PIL Image or Tensor

torchvision.transforms.functional.equalize(*img: torch.Tensor*) → torch.Tensor

[SOURCE]

Equalize the histogram of an image by applying a non-linear mapping to the input in order to create a uniform distribution of grayscale values in the output.

- Parameters:
- img** (*PIL Image or Tensor*) – Image on which equalize is applied. If img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, where ... means it can have an arbitrary number of leading dimensions. If img is PIL Image, it is expected to be in mode “P”, “L” or “RGB”.
- Returns:
- An image that was equalized.
- Return type:
- PIL Image or Tensor

torchvision.transforms.functional.erase(*img: torch.Tensor, i: int, j: int, h: int, w: int, v: torch.Tensor, inplace: bool = False*) → torch.Tensor

[SOURCE]

Erase the input Tensor Image with given value. This transform does not support PIL Image.

- Parameters:
- **img** (*Tensor Image*) – Tensor image of size (C, H, W) to be erased
 - **i** (*int*) – i in (i,j) i.e coordinates of the upper left corner.
 - **j** (*int*) – j in (i,j) i.e coordinates of the upper left corner.
 - **h** (*int*) – Height of the erased region.
 - **w** (*int*) – Width of the erased region.
 - **v** – Erasing value.
 - **inplace** (*bool, optional*) – For in-place operations. By default is set False.

Returns: Erased image.

Return type: Tensor Image

torchvision.transforms.functional.five_crop(*img: torch.Tensor, size: List[int]*) → Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

[SOURCE]

Crop the given image into four corners and the central crop. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

• NOTE

This transform returns a tuple of images and there may be a mismatch in the number of inputs and targets your Dataset returns.

- Parameters:
- **img** (*PIL Image or Tensor*) – Image to be cropped.
 - **size** (*sequence or int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).

Returns:

tuple (tl, tr, bl, br, center)

Corresponding top left, top right, bottom left, bottom right and center crop.

Return type: tuple

torchvision.transforms.functional.gaussian_blur(*img: torch.Tensor, kernel_size: List[int], sigma: Union[List[float], NoneType] = None*) → torch.Tensor

[SOURCE]

Performs Gaussian blurring on the image by given kernel. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **img** (*PIL Image or Tensor*) – Image to be blurred
 - **kernel_size** (*sequence of python:ints or int*) – Gaussian kernel size. Can be a sequence of integers like (kx, ky) or a single integer for square kernels. In torchscript mode kernel_size as single int is not supported, use a sequence of length 1: [ksize,] .
 - **sigma** (*sequence of python:floats or float, optional*) – Gaussian kernel standard deviation. Can be a sequence of floats like (sigma_x, sigma_y) or a single float to define the same sigma in both X/Y directions. If None, then it is computed using kernel_size as sigma = 0.3 * ((kernel_size - 1) * 0.5 - 1) + 0.8 . Default, None. In torchscript mode sigma as single float is not supported, use a sequence of length 1: [sigma,] .

Returns: Gaussian Blurred version of the image.

Return type: PIL Image or Tensor

torchvision.transforms.functional.hflip(*img: torch.Tensor*) → torch.Tensor

[SOURCE]

Horizontally flip the given image.

Parameters:
Returns:
Return type:

img (*PIL Image or Tensor*) – Image to be flipped. If img is a Tensor, it is expected to be in [..., H, W] format, where ... means it can have an arbitrary number of leading dimensions.
Horizontally flipped image.
PIL Image or Tensor

torchvision.transforms.functional.invert(*img: torch.Tensor*) → torch.Tensor

[SOURCE]

Invert the colors of an RGB/grayscale image.

Parameters:
Returns:
Return type:

img (*PIL Image or Tensor*) – Image to have its colors inverted. If img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, where ... means it can have an arbitrary number of leading dimensions. If img is PIL Image, it is expected to be in mode “L” or “RGB”.
Color inverted image.
PIL Image or Tensor

torchvision.transforms.functional.normalize(*tensor: torch.Tensor, mean: List[float], std: List[float], inplace: bool = False*) → torch.Tensor

[SOURCE]

Normalize a tensor image with mean and standard deviation. This transform does not support PIL Image.

• NOTE

This transform acts out of place by default, i.e., it does not mutates the input tensor.

See [Normalize](#) for more details.

Parameters:
Returns:
Return type:

- tensor** (*Tensor*) – Tensor image of size (C, H, W) or (B, C, H, W) to be normalized.
- mean** (*sequence*) – Sequence of means for each channel.
- std** (*sequence*) – Sequence of standard deviations for each channel.
- inplace** (*bool, optional*) – Bool to make this operation inplace.

Normalized Tensor image.
Tensor

torchvision.transforms.functional.pad(*img: torch.Tensor, padding: List[int], fill: int = 0, padding_mode: str = 'constant'*) → torch.Tensor

[SOURCE]

Pad the given image on all sides with the given “pad” value. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means at most 2 leading dimensions for mode reflect and symmetric, at most 3 leading dimensions for mode edge, and an arbitrary number of leading dimensions for mode constant

Parameters:
Returns:
Return type:

- img** (*PIL Image or Tensor*) – Image to be padded.
- padding** (*int or sequence*) – Padding on each border. If a single int is provided this is used to pad all borders. If sequence of length 2 is provided this is the padding on left/right and top/bottom respectively. If a sequence of length 4 is provided this is the padding for the left, top, right and bottom borders respectively. In torchscript mode padding as single int is not supported, use a sequence of length 1: `[padding, ]`.
- fill** (*number or str or tuple*) – Pixel fill value for constant fill. Default is 0. If a tuple of length 3, it is used to fill R, G, B channels respectively. This value is only used when the padding_mode is constant. Only number is supported for torch Tensor. Only int or str or tuple value is supported for PIL Image.
- padding_mode** –
Type of padding. Should be: constant, edge, reflect or symmetric. Default is constant.
 - constant: pads with a constant value, this value is specified with fill
 - edge: pads with the last value on the edge of the image,

if input a 5D torch Tensor, the last 3 dimensions will be padded instead of the last 2
 - reflect: pads with reflection of image (without repeating the last value on the edge)

padding [1, 2, 3, 4] with 2 elements on both sides in reflect mode will result in [3, 2, 1, 2, 3, 4, 3, 2]
 - symmetric: pads with reflection of image (repeating the last value on the edge)

padding [1, 2, 3, 4] with 2 elements on both sides in symmetric mode will result in [2, 1, 1, 2, 3, 4, 4, 3]

Padded image.
PIL Image or Tensor

torchvision.transforms.functional.perspective(*img: torch.Tensor, startpoints: List[List[int]], endpoints: List[List[int]], interpolation: torchvision.transforms.functional.InterpolationMode = <InterpolationMode.BILINEAR: 'bilinear'>, fill: Union[List[float], NoneType] = None*) → torch.Tensor

[SOURCE]

Perform perspective transform of the given image. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be transformed. • startpoints (<i>list of list of python:ints</i>) – List containing four lists of two integers corresponding to four corners [top-left, top-right, bottom-right, bottom-left] of the original image. • endpoints (<i>list of list of python:ints</i>) – List containing four lists of two integers corresponding to four corners [top-left, top-right, bottom-right, bottom-left] of the transformed image. • interpolation (<i>InterpolationMode</i>) – Desired interpolation enum defined by torchvision.transforms.InterpolationMode . Default is InterpolationMode.BILINEAR . If input is Tensor, only InterpolationMode.NEAREST , InterpolationMode.BILINEAR are supported. For backward compatibility integer values (e.g. PIL.Image.NEAREST) are still acceptable. • fill (<i>sequence or number, optional</i>) – Pixel fill value for the area outside the transformed image. If given a number, the value is used for all bands respectively. In torchscript mode single int/float value is not supported, please use a sequence of length 1: [value,] . If input is PIL Image, the options is only available for Pillow>=5.0.0 .
Returns:	transformed Image.
Return type:	PIL Image or Tensor

torchvision.transforms.functional.pil_to_tensor(pic)

[SOURCE]

Convert a PIL Image to a tensor of the same type. This function does not support torchscript.

See PILToTensor for more details.

Parameters:	pic (<i>PIL Image</i>) – Image to be converted to tensor.
Returns:	Converted image.
Return type:	Tensor

torchvision.transforms.functional.posterize(img: torch.Tensor, bits: int) → torch.Tensor

[SOURCE]

Posterize an image by reducing the number of bits for each color channel.

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to have its colors posterized. If img is torch Tensor, it should be of type torch.uint8 and it is expected to be in [..., 1 or 3, H, W] format, where ... means it can have an arbitrary number of leading dimensions. If img is PIL Image, it is expected to be in mode “L” or “RGB”. • bits (<i>int</i>) – The number of bits to keep for each channel (0-8).
Returns:	Posterized image.
Return type:	PIL Image or Tensor

torchvision.transforms.functional.resize(img: torch.Tensor, size: List[int], interpolation: torchvision.transforms.functional.InterpolationMode = <InterpolationMode.BILINEAR: 'bilinear'>) → torch.Tensor

[SOURCE]

Resize the input image to the given size. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be resized. • size (<i>sequence or int</i>) – Desired output size. If size is a sequence like (h, w), the output size will be matched to this. If size is an int, the smaller edge of the image will be matched to this number maintaining the aspect ratio. i.e, if height > width, then image will be rescaled to $\left(\text{size} \times \frac{\text{height}}{\text{width}}, \text{size}\right)$. In torchscript mode size as single int is not supported, use a sequence of length 1: [size,] . • interpolation (<i>InterpolationMode</i>) – Desired interpolation enum defined by torchvision.transforms.InterpolationMode . Default is InterpolationMode.BILINEAR . If input is Tensor, only InterpolationMode.NEAREST , InterpolationMode.BILINEAR and InterpolationMode.BICUBIC are supported. For backward compatibility integer values (e.g. PIL.Image.NEAREST) are still acceptable.
Returns:	Resized image.
Return type:	PIL Image or Tensor

torchvision.transforms.functional.resized_crop(img: torch.Tensor, top: int, left: int, height: int, width: int, size: List[int], interpolation: torchvision.transforms.functional.InterpolationMode = <InterpolationMode.BILINEAR: 'bilinear'>) → torch.Tensor

[SOURCE]

Crop the given image and resize it to desired size. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

Notably used in RandomResizedCrop .

Parameters:	• img (<i>PIL Image or Tensor</i>) – Image to be cropped. (0,0) denotes the top left corner of the image. • top (<i>int</i>) – Vertical component of the top left corner of the crop box. • left (<i>int</i>) – Horizontal component of the top left corner of the crop box. • height (<i>int</i>) – Height of the crop box. • width (<i>int</i>) – Width of the crop box. • size (<i>sequence or int</i>) – Desired output size. Same semantics as resize . • interpolation (<i>InterpolationMode</i>) – Desired interpolation enum defined by torchvision.transforms.InterpolationMode . Default is InterpolationMode.BILINEAR . If input is Tensor, only InterpolationMode.NEAREST , InterpolationMode.BILINEAR and InterpolationMode.BICUBIC are supported. For backward compatibility integer values (e.g. PIL.Image.NEAREST) are still acceptable.
Returns:	Cropped image.
Return type:	PIL Image or Tensor

torchvision.transforms.functional.rgb_to_grayscale(img: torch.Tensor, num_output_channels: int = 1) → torch.Tensor

[SOURCE]

Convert RGB image to grayscale version of image. If the image is torch Tensor, it is expected to have [..., 3, H, W] shape, where ... means an arbitrary number of leading dimensions

• NOTE

Please, note that this method supports only RGB images as input. For inputs in other color spaces, please, consider using meth:~torchvision.transforms.functional.to_grayscale with PIL Image.

- Parameters:
- **img** (*PIL Image or Tensor*) – RGB Image to be converted to grayscale.
 - **num_output_channels** (*int*) – number of channels of the output image. Value can be 1 or 3. Default, 1.

Returns:

Grayscale version of the image.

if num_output_channels = 1 : returned image is single channel

if num_output_channels = 3 : returned image is 3 channel with r = g = b

Return type:

PIL Image or Tensor

```
torchvision.transforms.functional.rotate(img: torch.Tensor, angle: float, interpolation:
torchvision.transforms.functional.InterpolationMode = <InterpolationMode.NEAREST: 'nearest'>, expand: bool = False, center:
Union[List[int], NoneType] = None, fill: Union[List[float], NoneType] = None, resample: Union[int, NoneType] = None) → torch.Tensor
```

[SOURCE]

Rotate the image by angle. If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions.

- Parameters:
- **img** (*PIL Image or Tensor*) – image to be rotated.
 - **angle** (*number*) – rotation angle value in degrees, counter-clockwise.
 - **interpolation** (*InterpolationMode*) – Desired interpolation enum defined by torchvision.transforms.InterpolationMode . Default is InterpolationMode.NEAREST . If input is Tensor, only InterpolationMode.NEAREST , InterpolationMode.BILINEAR are supported. For backward compatibility integer values (e.g. PIL .Image.NEAREST) are still acceptable.
 - **expand** (*bool, optional*) – Optional expansion flag. If true, expands the output image to make it large enough to hold the entire rotated image. If false or omitted, make the output image the same size as the input image. Note that the expand flag assumes rotation around the center and no translation.
 - **center** (*sequence, optional*) – Optional center of rotation. Origin is the upper left corner. Default is the center of the image.
 - **fill** (*sequence or number, optional*) – Pixel fill value for the area outside the transformed image. If given a number, the value is used for all bands respectively. In torchscript mode single int/float value is not supported, please use a sequence of length 1: [value,] . If input is PIL Image, the options is only available for Pillow>=5.2.0 .

Returns:

Rotated image.

Return type:

PIL Image or Tensor

```
torchvision.transforms.functional.solarize(img: torch.Tensor, threshold: float) → torch.Tensor
```

[SOURCE]

Solarize an RGB/grayscale image by inverting all pixel values above a threshold.

- Parameters:
- **img** (*PIL Image or Tensor*) – Image to have its colors inverted. If img is torch Tensor, it is expected to be in [..., 1 or 3, H, W] format, where ... means it can have an arbitrary number of leading dimensions. If img is PIL Image, it is expected to be in mode “L” or “RGB”.
 - **threshold** (*float*) – All pixels equal or above this value are inverted.

Returns:

Solarized image.

Return type:

PIL Image or Tensor

```
torchvision.transforms.functional.ten_crop(img: torch.Tensor, size: List[int], vertical_flip: bool = False) → List[torch.Tensor]
```

[SOURCE]

Generate ten cropped images from the given image. Crop the given image into four corners and the central crop plus the flipped version of these (horizontal flipping is used by default). If the image is torch Tensor, it is expected to have [..., H, W] shape, where ... means an arbitrary number of leading dimensions

• NOTE

This transform returns a tuple of images and there may be a mismatch in the number of inputs and targets your Dataset returns.

- Parameters:
- **img** (*PIL Image or Tensor*) – Image to be cropped.
 - **size** (*sequence or int*) – Desired output size of the crop. If size is an int instead of sequence like (h, w), a square crop (size, size) is made. If provided a sequence of length 1, it will be interpreted as (size[0], size[0]).
 - **vertical_flip** (*bool*) – Use vertical flipping instead of horizontal

Returns:

tuple (tl, tr, bl, br, center, tl_flip, tr_flip, bl_flip, br_flip, center_flip)

Corresponding top left, top right, bottom left, bottom right and center crop and same for the flipped image.

Return type:

tuple

```
torchvision.transforms.functional.to_grayscale(img, num_output_channels=1)
```

[SOURCE]

Convert PIL image of any mode (RGB, HSV, LAB, etc) to grayscale version of image. This transform does not support torch Tensor.

Parameters:	img (<i>PIL Image</i>) – PIL Image to be converted to grayscale. num_output_channels (<i>int</i>) – number of channels of the output image. Value can be 1 or 3. Default is 1.
Returns:	Grayscale version of the image. if num_output_channels = 1 : returned image is single channel if num_output_channels = 3 : returned image is 3 channel with r = g = b
Return type:	PIL Image

torchvision.transforms.functional.to_pil_image(pic, mode=None)

[SOURCE]

Convert a tensor or an ndarray to PIL Image. This function does not support torchscript.

See [ToPILImage](#) for more details.

Parameters:	pic (<i>Tensor or numpy.ndarray</i>) – Image to be converted to PIL Image. mode (<i>PIL.Image mode</i>) – color space and pixel depth of input data (optional).
Returns:	Image converted to PIL Image.
Return type:	PIL Image

torchvision.transforms.functional.to_tensor(pic)

[SOURCE]

Convert a PIL Image or numpy.ndarray to tensor. This function does not support torchscript.

See [ToTensor](#) for more details.

Parameters:	pic (<i>PIL Image or numpy.ndarray</i>) – Image to be converted to tensor.
Returns:	Converted image.
Return type:	Tensor

torchvision.transforms.functional.vflip(img: torch.Tensor) → torch.Tensor

[SOURCE]

Vertically flip the given image.

Parameters:	img (<i>PIL Image or Tensor</i>) – Image to be flipped. If img is a Tensor, it is expected to be in [..., H, W] format, where ... means it can have an arbitrary number of leading dimensions.
Returns:	Vertically flipped image.
Return type:	PIL Image or Tensor

Docs

Access comprehensive developer documentation for PyTorch

[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers

[View Tutorials](#)

Resources

Find development resources and get your questions answered

[View Resources](#)